

Lodestone

A planning tool for the discerning supervillain.

dbt + DuckDB

S2 spatial

FastAPI on Lambda

60-minute walkthrough

01 · PRODUCT

Why this ...

45,716

rows in the NASA dataset

Lots of where and what.

But only noise about the future. (it's probably a typo)

**A what-would-have-happened
simulator, not a predictor.**

History gives us where and what.

The supervillain placing magnets asks: what would have happened? (And then all I need is a time machine)

Defensible question for the data we actually have.

Small surface, serious plumbing.

Shape of the next hour

01 Product

you are here

02 Architecture + 5 demo stops

browser ↔ slides, woven

03 Code overview

as if it were Monday

04 AI workflow & retrospectives

what the brief asks for

05 Q & A

interrupt before then too

03 · PRODUCT

Four features, one defensible idea

01

Map of every landing

38,399 with valid coords, clustered.

02

S2 density heatmap

L3 → L7, user-selectable.

03

Magnet placement

S / M / L radii, dropped on the map.

04

Live yield

Catches · mass · class · iron yield.

Iron yield = mass × per-class metal fraction. The data layer doing meaningful work.

04 · ARCHITECTURE

Top-level shape



INFRASTRUCTURE

- S3 + CloudFront — static frontend
- Terraform — local state, GitHub Actions deploys
- ECR — image registry
- Lambda Function URL — response streaming

05 · BEAT 1 · DATA

DuckDB + dbt, locally and in CI

~100 ms

DuckDB cold open

45,716

rows, in memory

26

data tests in CI

2

dbt marts

MARTS

meteorites (fact)

meteorites_by_s2

REJECTED

Postgres

overkill, adds managed DB to bill

SQLite

DuckDB's columnar engine wins on aggs

Quack

client-server adds ops; single-reader

DEMO browser ~90s → world zoom → Sahara → marker popup → back to slides

S2 over H3, for one reason: hierarchy

S2 · chosen

Spherical quadtree

- Exactly 4 children per cell
- Children tile parent precisely
- Parent is a bit-shift on the cell ID
- Sum-of-children == parent → exact rollups

H3 · not chosen

Hexagonal grid (Uber)

- Has cellToParent / cellToChildren
- Children overlap into neighbours
- 12 pentagons per resolution, special-cased
- For aggregation: approximate, not exact

EXACT-ROLLUP PROOF

$$\text{afc} = 7,026 = \text{af9} (6,826) + \text{aff} (200)$$

One SQL statement, three defensible choices

backend/db.py · yield query

```
SELECT COUNT(DISTINCT m.id) AS catches,
       SUM(m.mass)          AS mass,
       SUM(m.mass * c.iron_pct) AS iron_yield
FROM meteorites m, magnets mag
JOIN class_iron c USING (class)
WHERE 6371 * acos(
    LEAST(1.0, sin(...)*sin(...) + cos(...))
) <= mag.radius_km;
```

DISTINCT on meteorite id

Outer dedup — overlapping magnets don't double-count.

LEAST(1.0, ...) clamp

Float rounding can push dot product just above 1.0;
acos throws a domain error without the clamp.

acos form, not asin

Algebraically equivalent; one fewer op.

DEMO *L magnet over Antarctica → M over Sahara → S overlapping M (no double-count)*

FastAPI on Lambda, no Mangum shim

FastAPI + uvicorn

Async-first, auto OpenAPI, Pydantic models.

Container image on Lambda

10 GB limit vs 250 MB zip. Room for DuckDB + deps + data.

Lambda Web Adapter

Sidecar in the container. No Mangum(app), no handler(event, context). Same `uvicorn main:app` runs locally and in prod.

DuckDB read-only at warm-up

No CSV parsing at runtime. Warehouse is pre-built into the image.

Function URL · RESPONSE_STREAM

Ready for the disabled agent's SSE.

React + Vite + Leaflet, with one cache trick

React

Vite

Tailwind

react-leaflet

react-leaflet-cluster

Heatmap eager-fetch

All 5 S2 levels in parallel on first toggle — ~1 MB, ~1 sec.
Level switching afterwards is instant; everything cached client-side.

No client-side S2 library

Backend ships polygon vertices with every cell.
Leaflet just draws polygons; no JS-side S2 dependency.

Considered & rejected

deck.gl WebGL rendering of all 45k points without clustering.
Would have solved a problem I didn't have.

Vite asset hashing

Content-hashed filenames let me cache /assets/* immutable
and no-cache index.html. Two-pass aws s3 sync handles it.

Terraform → AWS, with two bug stories

TERRAFORM-MANAGED

ECR · Lambda · Function URL · IAM · S3 · CloudFront

CI · 8 pipelines ·
path-filtered

Local state — intentionally.

lint · pytest · dbt · tf-plan · deploy

Issue · best vs good enough

AWS wanted full managed state

Original proposal had textbook "best practice" S3 state bucket + DynamoDB locks. But the script kept extending doing the bucket/lock-table dance. For this scale - easier to manage local state.

BUG · silent 403s from Function URL

AWS changed the Function URL permission model in Oct 2025.

New URLs require two statements in the resource policy, not one. Most tutorials still show the old form. Cost an hour.

11 · CODE ONBOARDING

If I were onboarding you on Monday

 ~/meteorite-explorer

```
$ make install           # backend + frontend deps
$ make dbt-install       # dbt-duckdb + s2sphere
$ make dbt-build         # builds data/meteorites.duckdb – runs 26 tests
$ make dev-backend       # FastAPI on :8000
$ make dev-frontend      # Vite on :5173 – proxies /api → :8000
```

dbt-build is the long one (~30s) — runs the pipeline + 26 tests. After that, hot-reload all the way down.

12 · CODE TOUR

The S2 mart — dbt Python model

```
dbt/models/marts/meteorites_by_s2.py
```

Python model, not SQL

Needed s2sphere — not available in pure SQL.

One DataFrame, 5 levels

Levels L3 → L7 concatenated; level column distinguishes.

Cell metadata in the row

Computes centroid + 4-vertex polygon boundary per cell.
Frontend draws polygons — no S2 lib needed client-side.

Cell IDs as hex tokens

S2 int64s break JS Number above 2^{53} . Hex strings survive.

IDE *open the file, scroll to the level-loop, talk over it*

13 · CODE TOUR

The yield query

backend/db.py

mag CTE

Magnets come in as a CTE, joined against the fact table — that's the whole query shape.

LEAST(1.0, ...)

Defensive clamp inside acos. Prevents float-rounding domain errors when two points are very close.

DISTINCT

Outer DISTINCT on meteorite id. Dedupe across overlapping magnets — once per rock, regardless of catch count.

acos form

Algebraically equivalent to asin haversine. One fewer op per row at 45k rows.

Routes, the map, and CI

backend/main.py

Routes & schemas

Pydantic: MagnetIn, YieldOut.
Lifespan opens DuckDB once;
reused across requests.

frontend/src/components/Map.tsx

Leaflet integration

Markers → MarkerClusterGroup.
Polygons → LayerGroup,
toggled with the heatmap switch.

.github/workflows/

5 pipelines, path-filtered

lint · pytest · dbt build
tf plan · two deploys.
Frontend PR ≠ Docker build.

Quick file tour in the IDE — 30 sec per file. Don't over-explain.

Three modes I used AI for

PLAN

Scaffolding & shape

Project layout, sketch the dbt model structure, S2 vs H3 explored conceptually.

RESEARCH

Resolving unknowns

Fastest way to find the Oct 2025 Function URL permission change; LWA vs Mangum tradeoff.

EXECUTE

First drafts

Initial dbt Python model, haversine SQL, Terraform resources. Every draft read, edited, sometimes overridden.

THE DISCIPLINE

AI drafts. I read, push back, commit.

- Never copied SQL into the repo without running it.
- Never accepted infra code without poking at each resource.
- Kept adding more tests and guardrails to protect us both

What I'd do differently with more time

01 Dev container

git clone + one command — no Python/Node/Docker/Terraform dance.

low effort

02 API Gateway

Route /api/* through API Gateway and /* through S3.
Adds throttling and per-route metrics out of the box.

infra

03 Postgres+PostGIS

Swap DuckDB's haversine table scan for a GiST-indexed GEOGRAPHY column and ST_DWithin — the yield query goes from $O(n)$ to $O(\log n)$.

at scale

04 Map filters in the UI

Class · year range · mass · fell-vs-found. Both marker and heatmap views.

feature

05 Best magnet placement algo

Have some fun trying to do some more “best fit” algorithm for a set number of magnets - possibly with some filter limitations!

major

Thanks.

Questions?

README

dbt docs

repo

BACKUP

Extra material, if questions go deep.

Not in the main flow.

- A · dbt docs site walkthrough
- B · terraform/README.md auto-generation
- C · Haversine math derivation
- D · Lambda cold start metrics

BACKUP A

dbt docs site walkthrough

Lineage graph

Sources → staging → marts. Visual proof the pipeline isn't a script.

Column descriptions

Every column documented; descriptions inherit through models.

Test coverage

All 26 tests visible per model. Click through to assertions.

Hosted on GitHub Pages – bookmarked as fallback if local docs server isn't up.

BACKUP B

Auto-generated terraform docs

terraform/

Auto-regenerated on every push to main via terraform-docs/gh-actions.

No human writes the resource table. The diff is the proof.

Providers · aws ~> 5.0

Resources · ECR · Lambda · Function URL · IAM roles · S3 · CloudFront

Inputs · region · environment · domain_name

Outputs · function_url · cloudfront_domain · ecr_repository_url

Infra docs stay in sync because they're never written by hand.

BACKUP C

Haversine and the LEAST(1.0) clamp

asin form

$$d = 2R \cdot \text{asin}(\sqrt{(\sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2))})$$

acos form • used

$$d = R \cdot \text{acos}(\sin \phi_1 \cdot \sin \phi_2 + \cos \phi_1 \cdot \cos \phi_2 \cdot \cos(\Delta\lambda))$$

WHY THE CLAMP

acos is defined on [-1, 1].

For two points very close together the inner expression can rounding-error to 1.0000000001 and throw a domain error. LEAST(1.0, ...) clamps it. Distance returns zero — which is correct.

"Versine" = $1 - \cos \theta$ — the antique navigation function the formula is named for.

BACKUP D

Lambda cold start budget

LWA + uvicorn startup

Sidecar boot, FastAPI app import.

~ 400 ms

DuckDB open (read-only)

File-backed, no parse — warehouse pre-built.

~ 100 ms

Dataset load to memory

45k rows, columnar, one read.

~ 200 ms

First /health 200

After everything else is warm.

~ 50 ms

Total cold start budget

≈ 750 ms

Evaluated and rejected: duckdb_spatial

ST_Distance_Spheroid

Vincenty's formula · more accurate, in general

NaN at high latitudes

Vincenty's known convergence failure

(lon, lat) order

GIS convention

ST_Distance_Sphere

Spherical formula · equivalent to ours

Works at the poles

Spherical math has no convergence issue

(lat, lon) order

Opposite of Spheroid — same library

VERDICT

Keep the explicit haversine + **LEAST(1.0, ...)** clamp.